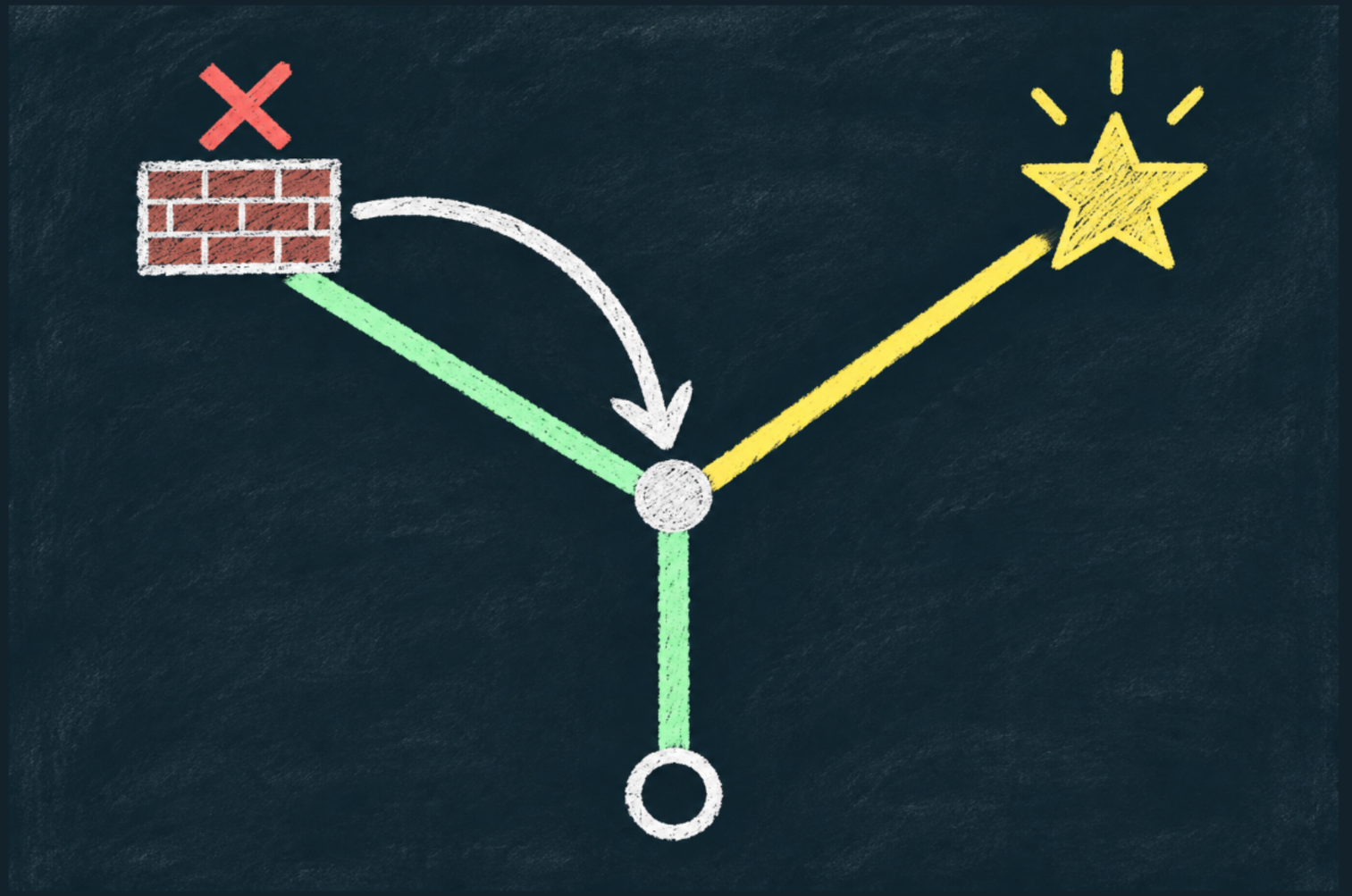
07

백트래킹

한 번 골라 보고, 돌아와서 다른 것을 골라 보기

막혔다면 마지막 선택으로 돌아옵니다



왼쪽 길을 골랐는데 막혀 있습니다.

갈림길로 돌아와 오른쪽 길을 가봅니다.

문제에서도 앞의 선택을 바꾸어 볼 수 있습니다.

이렇게 되돌아가며 찾는 방법이 백트래킹입니다.

도입 문제: 두 숫자를 문자로 바꾸기

전화기의 숫자 2에는 알파벳 a, b, c가, 숫자 5에는 j, k, l이 적혀 있다. 입력으로 두 자리 문자열 "25"가 주어진다.

2에서 첫 글자 하나를 고르고 5에서 둘째 글자 하나를 골라, 그 순서대로 이어 붙인다. 각 자리에서 반드시 한 글자씩 골라야 한다.

이 규칙으로 만들 수 있는 두 글자 문자열을 모두 구하시오.

예: "aj"는 가능하다. "ja"는 숫자 순서를 바꾸었으므로 불가능하다.

첫 자리는 2에서, 둘째 자리는 5에서



빈칸은 두 개입니다.

첫 칸: a, b, c 중 하나

둘째 칸: j, k, l 중 하나

어느 글자를 골랐는지 한 칸씩 기억합니다.

먼저 a로 시작하는 답을 만듭니다



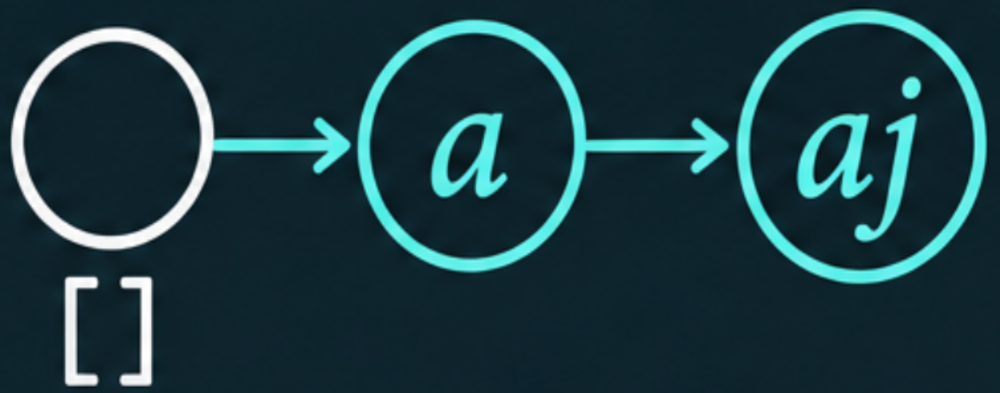
첫 글자로 a를 골랐습니다.

첫 칸의 a는 그대로 둡니다.

이제 둘째 칸에 넣을 j, k, l을 봅니다.

현재 만든 글자는 a입니다.

j를 붙이면 답 하나가 완성됩니다



a 뒤에 j를 붙였습니다.

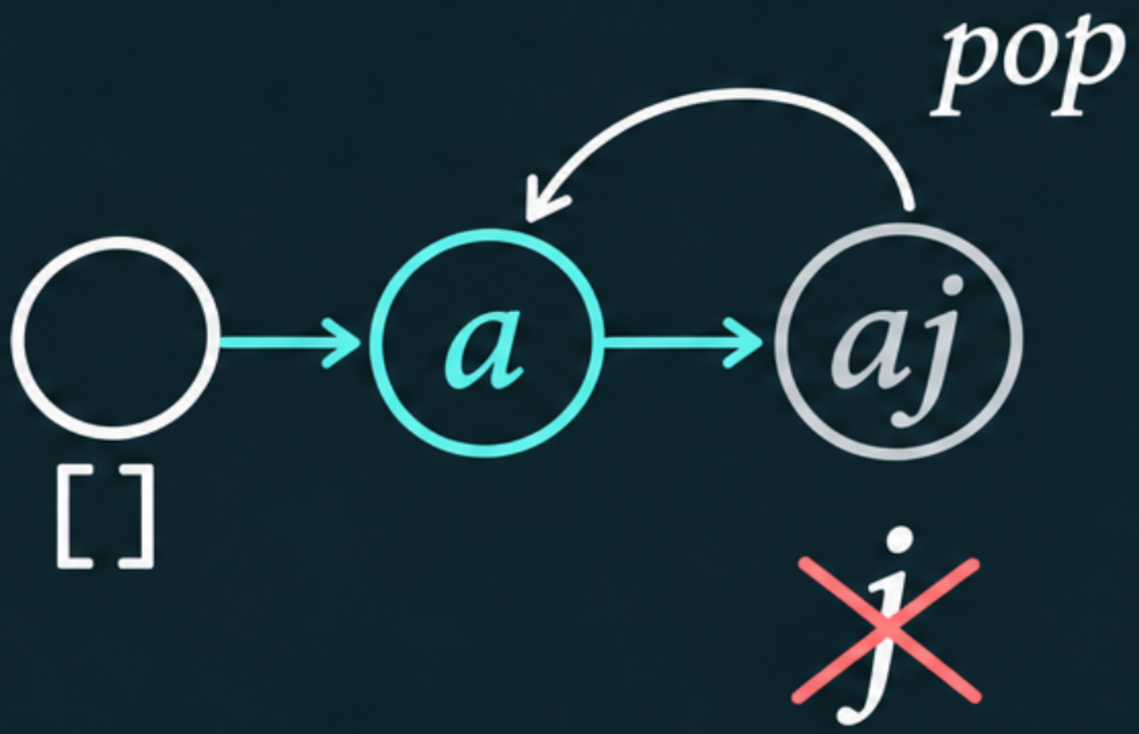
두 칸을 모두 채웠습니다.

완성한 aj를 답 목록에 적습니다.

aj

찾은 답: aj

마지막에 붙인 j만 지웁니다



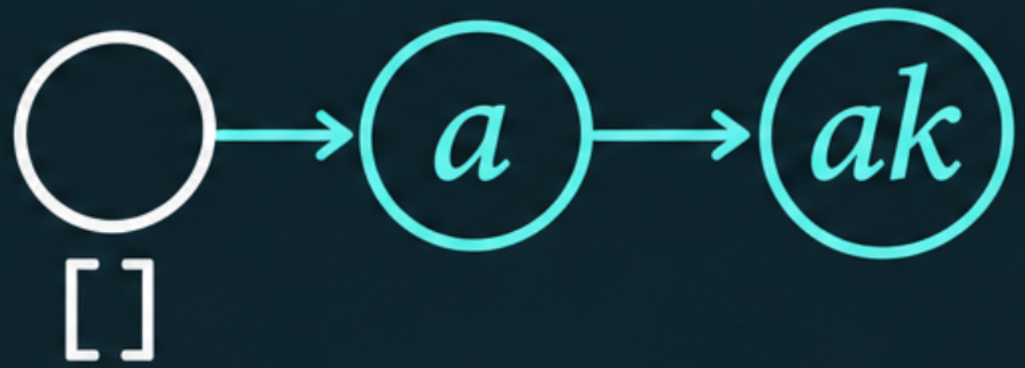
aj에서 j를 지웠습니다.

현재 글자는 다시 a입니다.

답 목록에 적은 aj는 지우지 않습니다.

둘째 칸을 고르기 전으로 돌아왔습니다.

그 자리에 이번에는 k를 넣습니다



aj ak

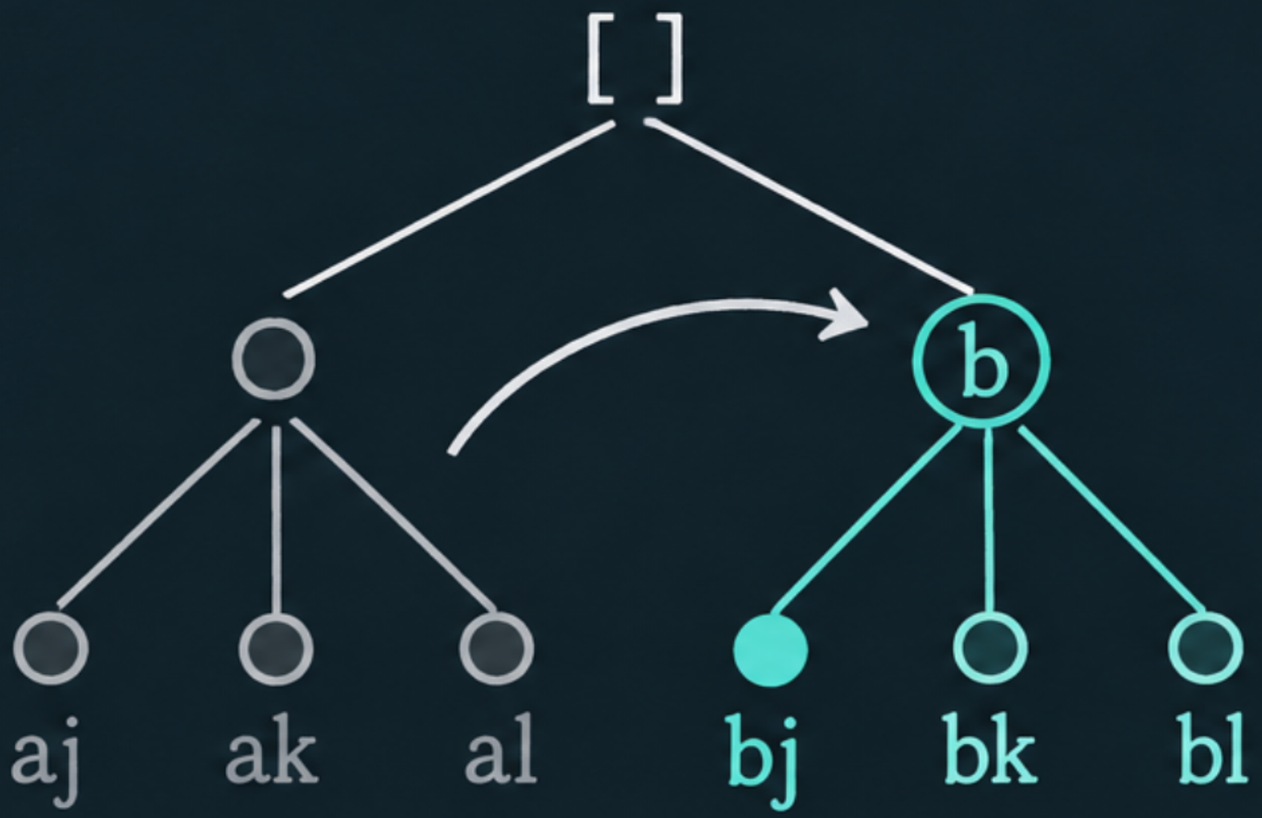
a 뒤에 k를 붙였습니다.

ak도 완성된 답입니다.

다시 k를 지우고 l을 붙일 수 있습니다.

a로 시작하는 답: aj, ak, al

a로 시작하는 답을 다 봤으면 b로 바꿉니다



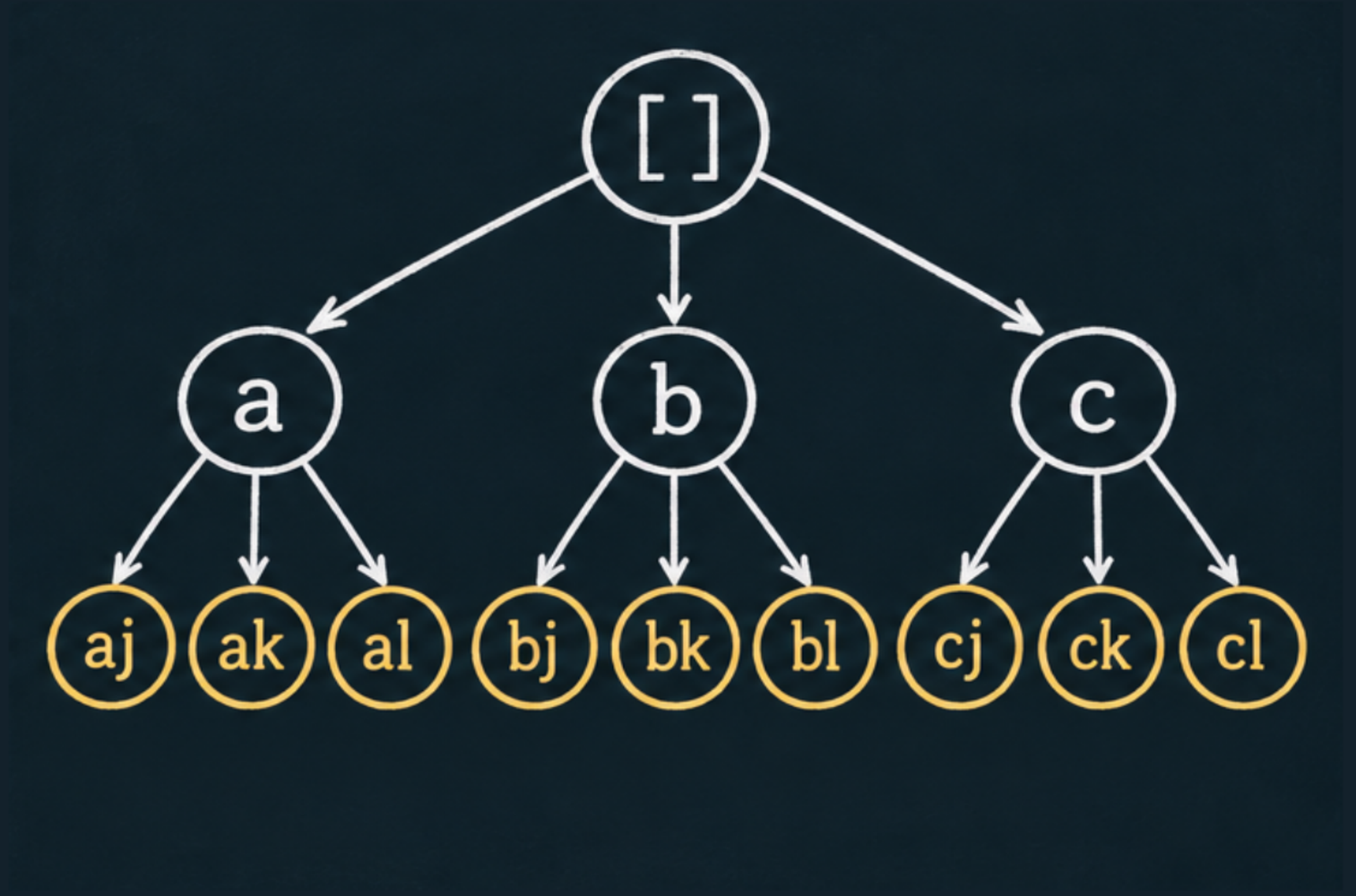
이제 첫 글자 a를 지웁니다.

첫 자리에 b를 놓습니다.

다시 둘째 자리 j, k, l을 고릅니다.

bj, bk, bl도 같은 방식으로 만듭니다.

가능한 답을 빠짐없이 찾았습니다



첫 글자마다 답 세 개가 있습니다.

a: aj, ak, al / b: bj, bk, bl

c: cj, ck, cl

총 아홉 문자열입니다.

공통 동작은 고르기와 되돌리기입니다

하나를 고른 뒤, 그 선택을 유지한 채 다음 자리를 처리합니다.

그 아래 선택을 모두 확인하면 방금 고른 것을 취소합니다.

같은 자리에서 다음 후보를 골라 다시 진행합니다.

이 순서를 코드로 옮기면 됩니다.

문제 7-2: 목표 합을 만드는 선택의 수

양의 정수 배열 a와 목표 합 target이 주어진다. a의 원소를 몇 개 골라 그 합을 target으로 만들려고 한다.

각 위치의 원소는 한 번만 고를 수 있다. 고른 순서는 구분하지 않지만, 같은 값이라도 서로 다른 위치의 원소를 골랐다면 다른 선택으로 센다. 아무것도 고르지 않은 선택도 포함하며 그 합은 0이다.

배열의 길이는 0~20, 각 원소는 1~50, target은 0~1000의 정수이다.

고른 원소의 합이 target인 선택의 수를 정수로 반환하는 subset_sum_count 함수를 작성하시오.

예: a=[1,2,3], target=3이면 [1,2]와 [3]의 두 선택이 있으므로 2를 반환한다.

Python: subset_sum_count(a, target) / C: subset_sum_count(a, n, target), n은 배열 길이

숫자를 골라 합을 3으로 만듭니다



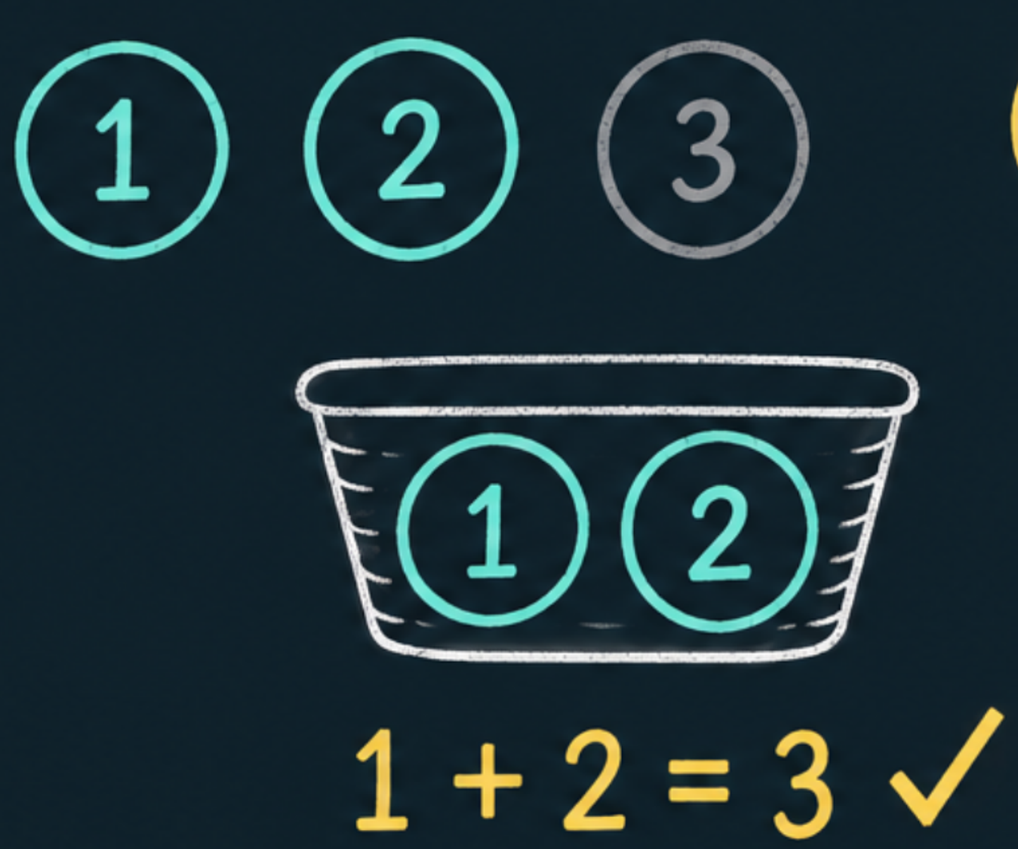
입력은 [1, 2, 3], 목표는 3입니다.

숫자마다 고르거나 건너뛸 수 있습니다.

어떤 숫자를 골랐는지 끝까지 확인합니다.

합이 3인 선택이 몇 개인지 셉니다.

1과 2를 고르면 첫 번째 성공입니다



고른 값은 [1, 2]입니다.

1+2=3이므로 조건에 맞습니다.

이 선택을 한 가지로 셉니다.

현재 성공한 선택 수: 1

3만 고르는 것도 두 번째 성공입니다



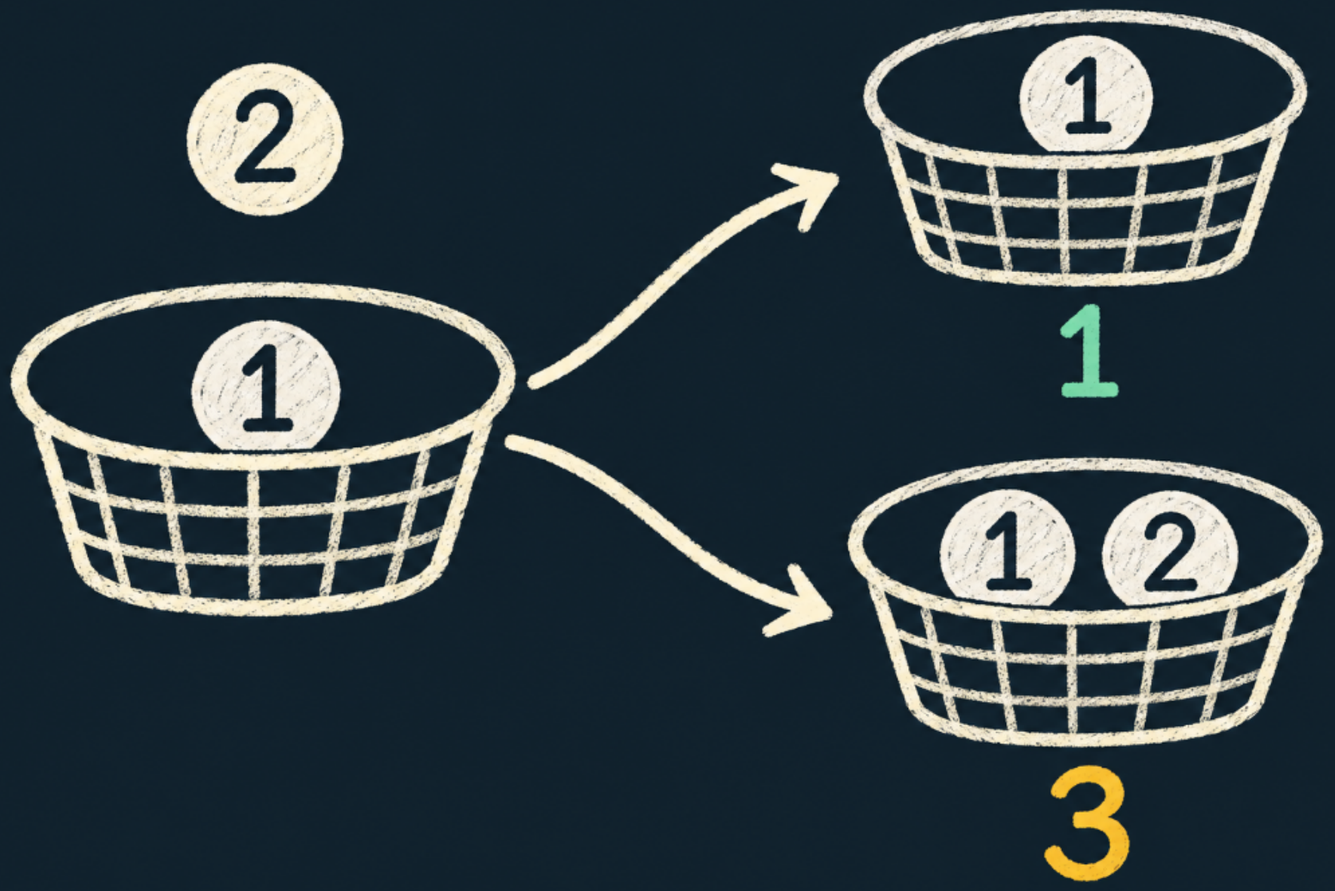
1과 2는 건너뛰고 3만 골랐습니다.

3 하나의 합도 3입니다.

[1,2]와 다른 선택입니다.

전체 답: 2

현재 숫자에서 할 일은 두 가지입니다



앞에서 1을 골랐고, 이제 2를 봅니다.

위: 2를 건너뛰면 합은 그대로 1

아래: 2를 고르면 합은 3

두 선택 모두 다음 숫자로 갑니다.

함수에 알려줄 값은 위치와 현재 합입니다

search(i, total): i번 숫자부터 결정하며, 지금까지 고른 합은 total입니다.

search(1, 1)은 "합이 1이고, 이제 1번 위치의 숫자 2를 볼 차례"입니다.

처음에는 아무 숫자도 고르지 않았으므로 search(0, 0)에서 시작합니다.

i는 위치이고, total은 고른 숫자의 합입니다.

건너뛸 때와 고를 때의 다음 호출

현재: search(1, 1)

건너뛰기: search(2, 1)
고르기: search(2, 3)

현재 보는 숫자는 a[1], 즉 2입니다.

건너뛰면 합 1을 유지합니다.

고르면 합에 2를 더합니다.
다음 위치는 둘 다 2입니다.

각 숫자의 선택을 한 번씩 정하므로, 선택 순서를 바꿔 중복해서 세지 않습니다.

숫자를 모두 보았을 때 성공 여부를 정합니다

i가 배열 길이 3이 되면, 세 숫자를 고를지 모두 결정한 상태입니다.

search(3, 3)은 합이 목표 3이므로 성공 한 가지를 뜻하는 1을 돌려줍니다.

합이 목표와 다르면 그 선택은 세지 않으므로 0을 돌려줍니다.

마지막에서 돌아오는 1과 0을 모아 성공 개수를 셉니다.

두 갈래에서 찾은 성공 수를 더합니다

건너뛴 쪽의 성공 수

+

고른 쪽의 성공 수

첫 숫자 1을 건너뛴 쪽에서는 [3] 한 가지가 나옵니다.

1을 고른 쪽에서는 [1,2] 한 가지가 나옵니다.

두 쪽의 결과 1+1을 더해 2를 반환합니다.

이 더하기는 원소의 합이 아니라, 찾아낸 선택의 개수를 더하는 것입니다.

다녀온 뒤에도 현재 호출의 합은 남아 있습니다

현재 합이 1일 때, 고르는 쪽에는 1+2=3을 계산해서 넘깁니다.

다음 호출이 3을 받았다고 해서 현재 호출의 total=1이 바뀌지는 않습니다.

따라서 다른 쪽을 계산할 때는 원래 합 1을 그대로 사용할 수 있습니다.

이 코드는 합을 숫자로 넘기므로, 돌아온 뒤 따로 빼는 줄이 없습니다.

합이 이미 목표를 넘었으면 그만 봅니다

```
a = [2, 4, 1], target = 3
2와 4를 고른 현재 합 = 6

1을 건너뛰면 6
1을 고르면   7
```

남은 숫자는 양수 1입니다.

고르든 건너뛰든 합이 3으로 줄어들 수 없습니다.

그래서 이 갈래의 성공 수는 0입니다.

더 진행해도 성공할 수 없는 갈래를 멈추는 것을 가지치기라고 합니다.

7-2 Python: 두 선택의 개수를 더하기

```python
def subset_sum_count(a, target):
    def search(i, total):
        if total > target:
            return 0
        if i == len(a):
            return int(total == target)
        return (search(i+1, total)
                + search(i+1, total+a[i]))
    return search(0, 0)
```

search(i, total)은 i번부터 확인해 만들 수 있는 성공 선택의 수를 반환합니다.

C에서도 같은 값을 함수에 넘깁니다

a는 입력 배열이고 n은 배열의 길이입니다. i는 다음 위치, total은 현재 합입니다.

subset_search는 탐색을 맡는 보조 함수입니다. 시작 함수보다 위에 따로 작성합니다.

subset_sum_count는 사이트가 부릅니다. 이 함수가 위치 0과 합 0으로 탐색을 시작합니다.

const는 배열을 읽기만 한다는 표시이고, static은 이 파일 안에서 쓰는 함수라는 표시입니다.

7-2 C: 시작 함수와 탐색 함수

```
1   static int subset_search(const int a[], int n, int target,
2                            int i, int total) {
3       if (total>target) return 0;
4       if (i==n) return total==target;
5       return subset_search(a,n,target,i+1,total)
6            + subset_search(a,n,target,i+1,total+a[i]);
7   }
8   int subset_sum_count(const int a[], int n, int target) {
9       return subset_search(a,n,target,0,0);
10  }
```

두 subset_search 호출이 찾아낸 성공 수를 더해 반환합니다.

이 입력들도 같은 규칙으로 읽을 수 있습니다

a=[1,1], target=1: 앞의 1만 고르거나 뒤의 1만 고르므로 2입니다.

a=[], target=0: 아무것도 고르지 않은 선택 하나가 맞으므로 1입니다.

a=[2,4], target=3: 가능한 합은 0, 2, 4, 6이므로 0입니다.

값이 같아도 위치가 다르면 다른 선택으로 셉니다.

문제 7-3: 모든 점을 조건에 맞게 색칠하기

n개의 점으로 이루어진 무방향 그래프를 k가지 색으로 칠하려고 한다. 모든 점에 색 하나를 칠하고, 선으로 직접 연결된 두 점의 색은 서로 달라야 한다.

연결 정보는 n×n 배열 graph에 주어진다. graph[i][j]=1이면 i번과 j번 점이 연결되고, 0이면 연결되지 않는다. 이 배열은 대칭이며 대각선 값은 0이다.

n은 1~8, k는 1~3의 정수이다. 주어진 색을 모두 쓸 필요는 없으며, 떨어져 있는 점도 빠짐없이 칠해야 한다.

이 조건으로 모든 점을 칠할 수 있으면 1, 없으면 0을 반환하는 can_color 함수를 작성하시오.

예: 세 점이 모두 서로 연결된 삼각형은 k=2이면 0, k=3이면 1을 반환한다.

Python: can_color(graph, k) / C: can_color(graph, n, k)

연결된 두 점은 표에 1로 적습니다

```
     0   1   2
0  [0,  1,  0]
1  [1,  0,  1]
2  [0,  1,  0]
```

위와 왼쪽의 숫자는 점 번호입니다.

graph[0][1] = 1
0번과 1번 사이에 선이 있습니다.

graph[0][2] = 0
0번과 2번 사이에는 선이 없습니다.

표의 각 칸은 "두 점 사이에 선이 있는가"를 나타냅니다.

선으로 직접 연결된 점끼리만 색이 달라야 합니다



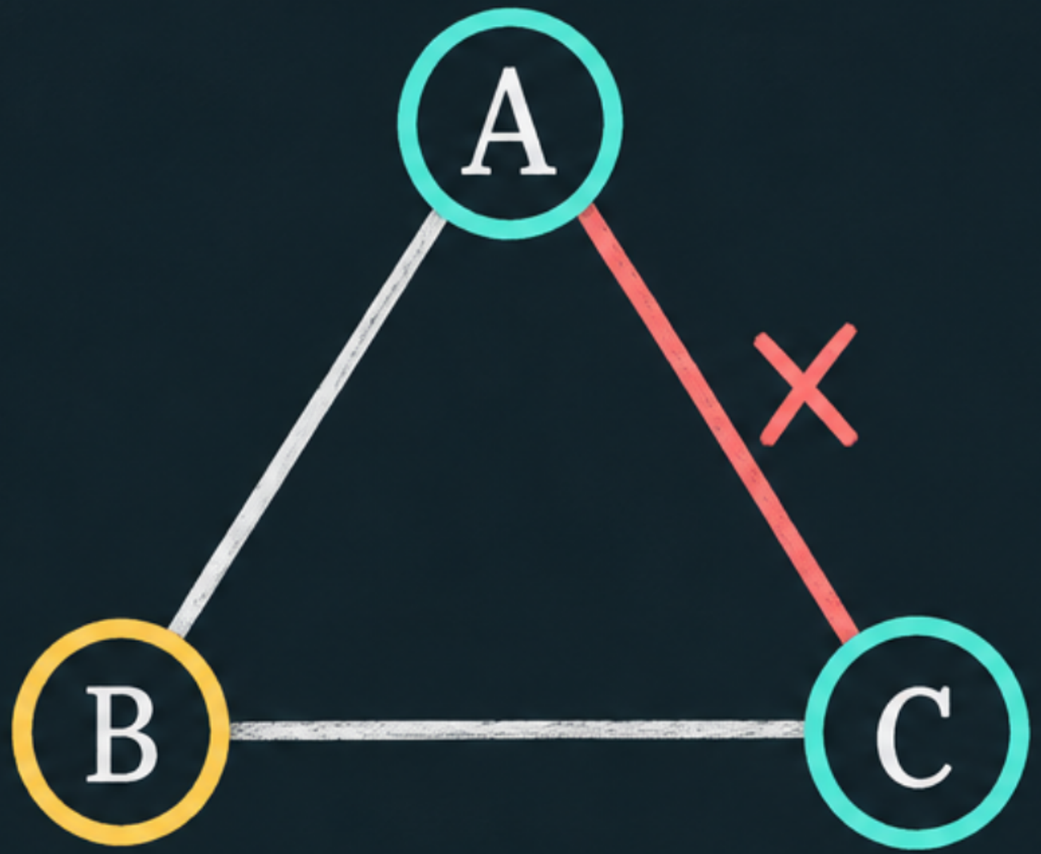
A=0번, B=1번, C=2번입니다.

A와 C 사이에는 선이 없습니다.

A와 C가 같은 초록이어도 됩니다.

두 색으로 모두 칠할 수 있으므로 답은 1

삼각형에서는 두 색이 모두 막힙니다



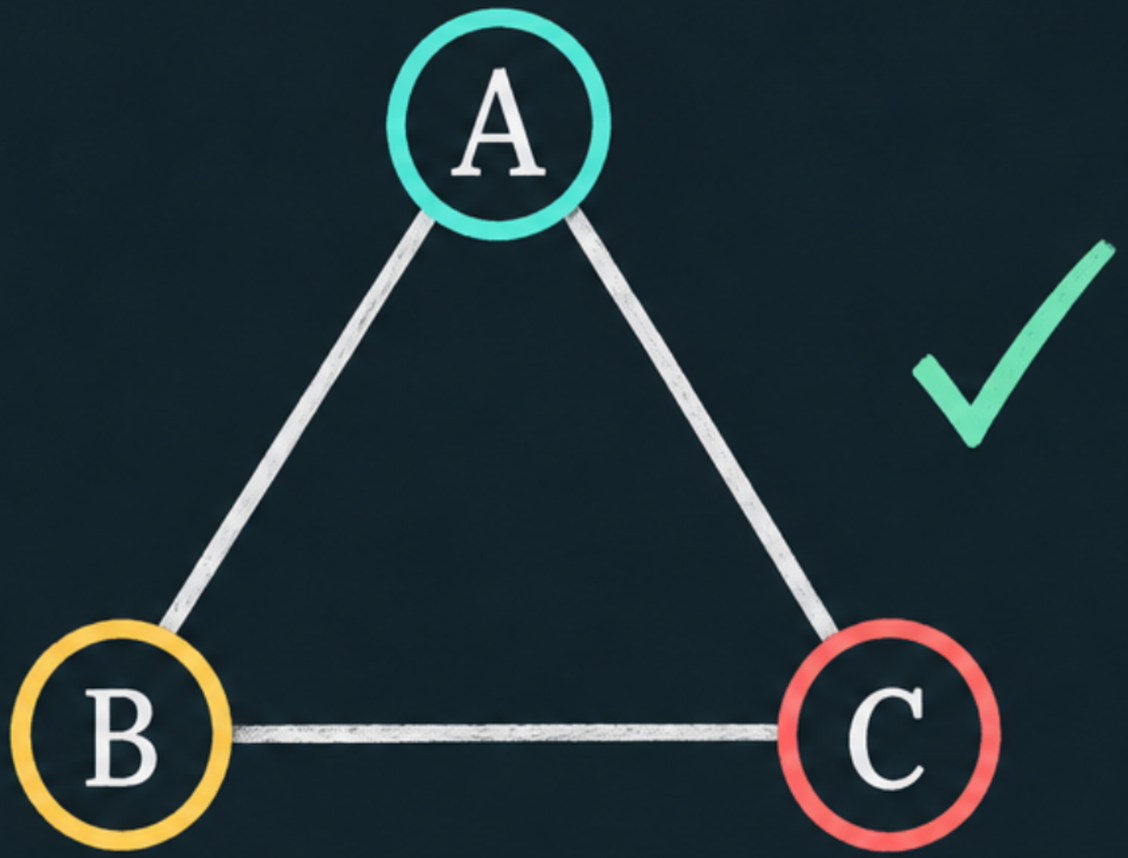
이번에는 세 점이 모두 연결되어 있습니다.

A가 초록, B가 노랑이라면

C는 초록도 노랑도 쓸 수 없습니다.

두 색으로는 불가능하므로 답은 0

세 번째 색이 있으면 삼각형도 가능합니다



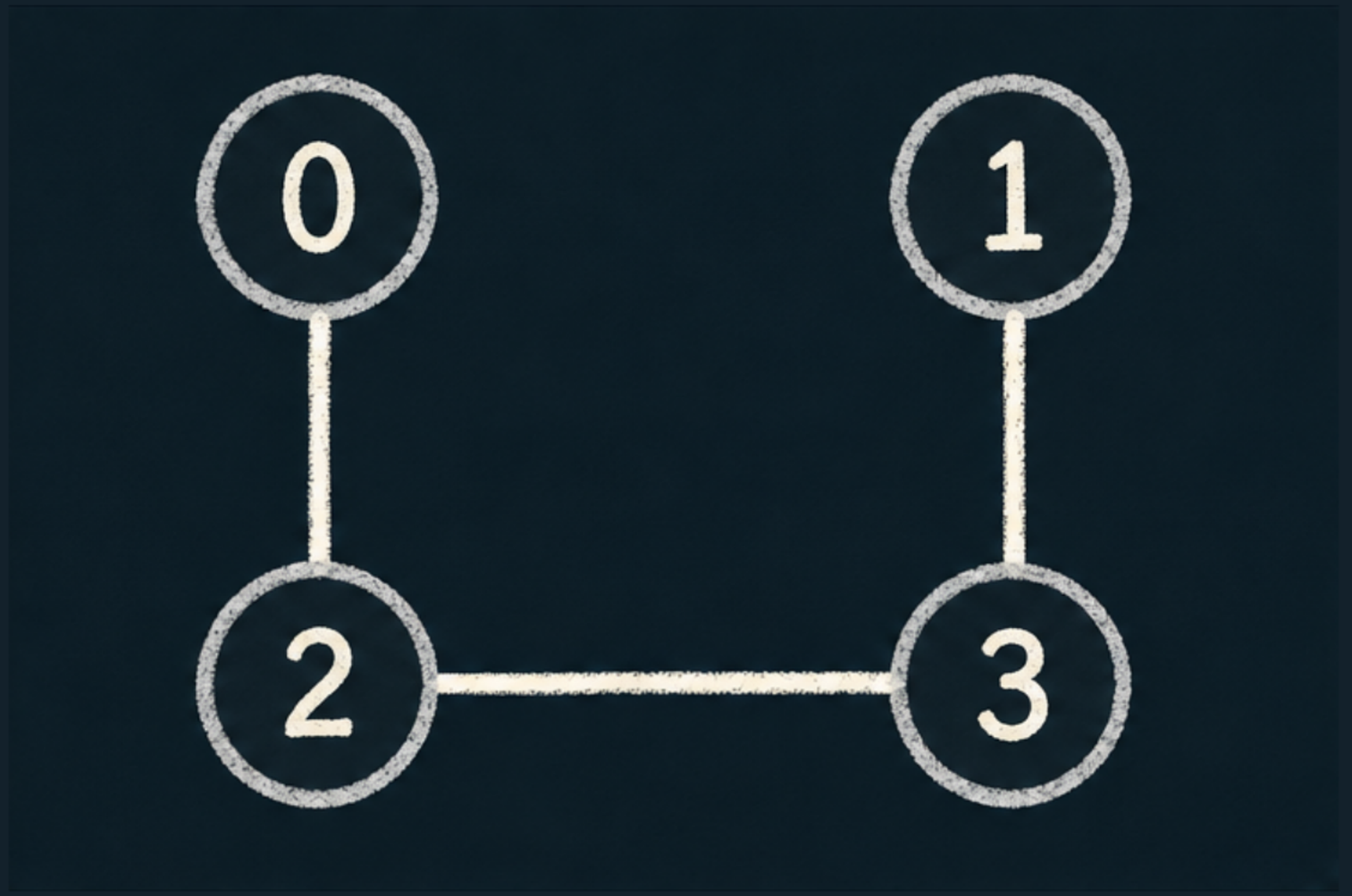
C에 세 번째 색을 칠합니다.

세 선의 양 끝 색이 모두 다릅니다.

모든 점을 칠했으므로 성공입니다.

k=3일 때의 답은 1

네 점을 두 색으로 칠해 봅시다



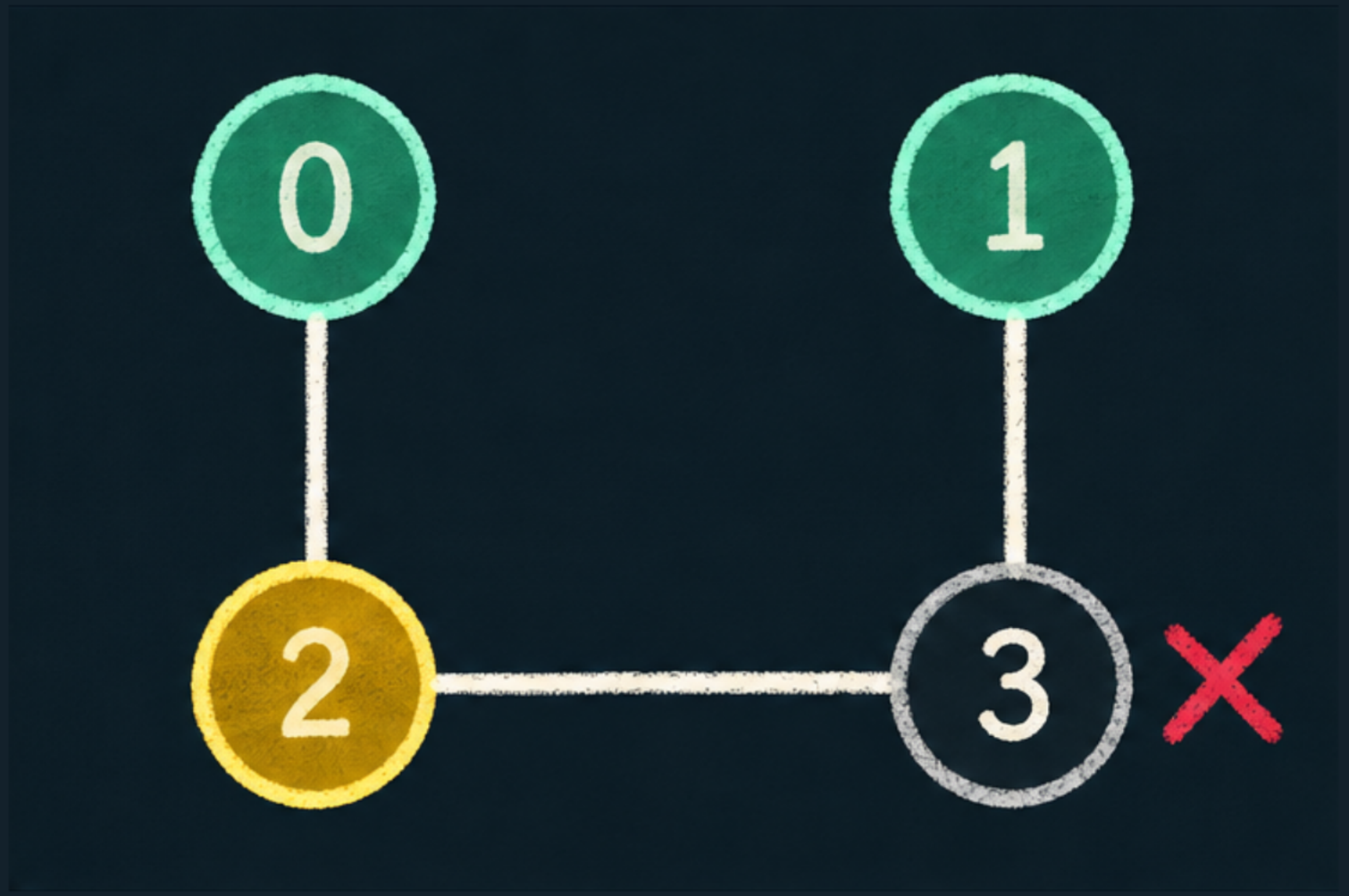
점 번호는 0, 1, 2, 3입니다.

선은 0–2, 2–3, 3–1에만 있습니다.

점 번호 순서대로 색을 정합니다.

초록과 노랑 두 색만 사용합니다.

마지막 점에서 두 색이 모두 막혔습니다



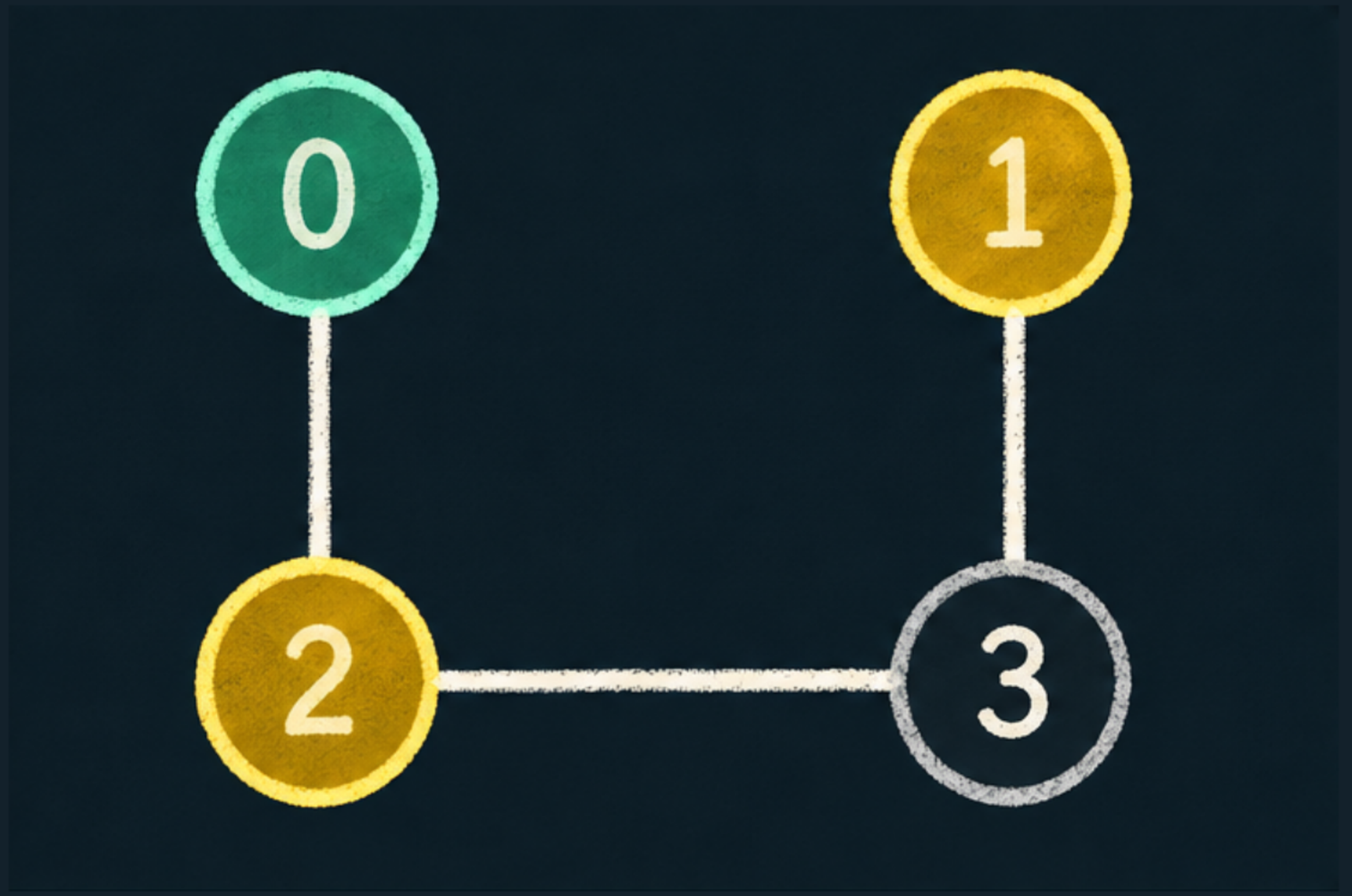
0번 초록, 1번 초록, 2번 노랑입니다.

3번을 초록으로 칠하면 1번과 충돌

3번을 노랑으로 칠하면 2번과 충돌

이 선택들로는 끝까지 갈 수 없습니다.

1번의 색을 지우고 노랑으로 바꿉니다



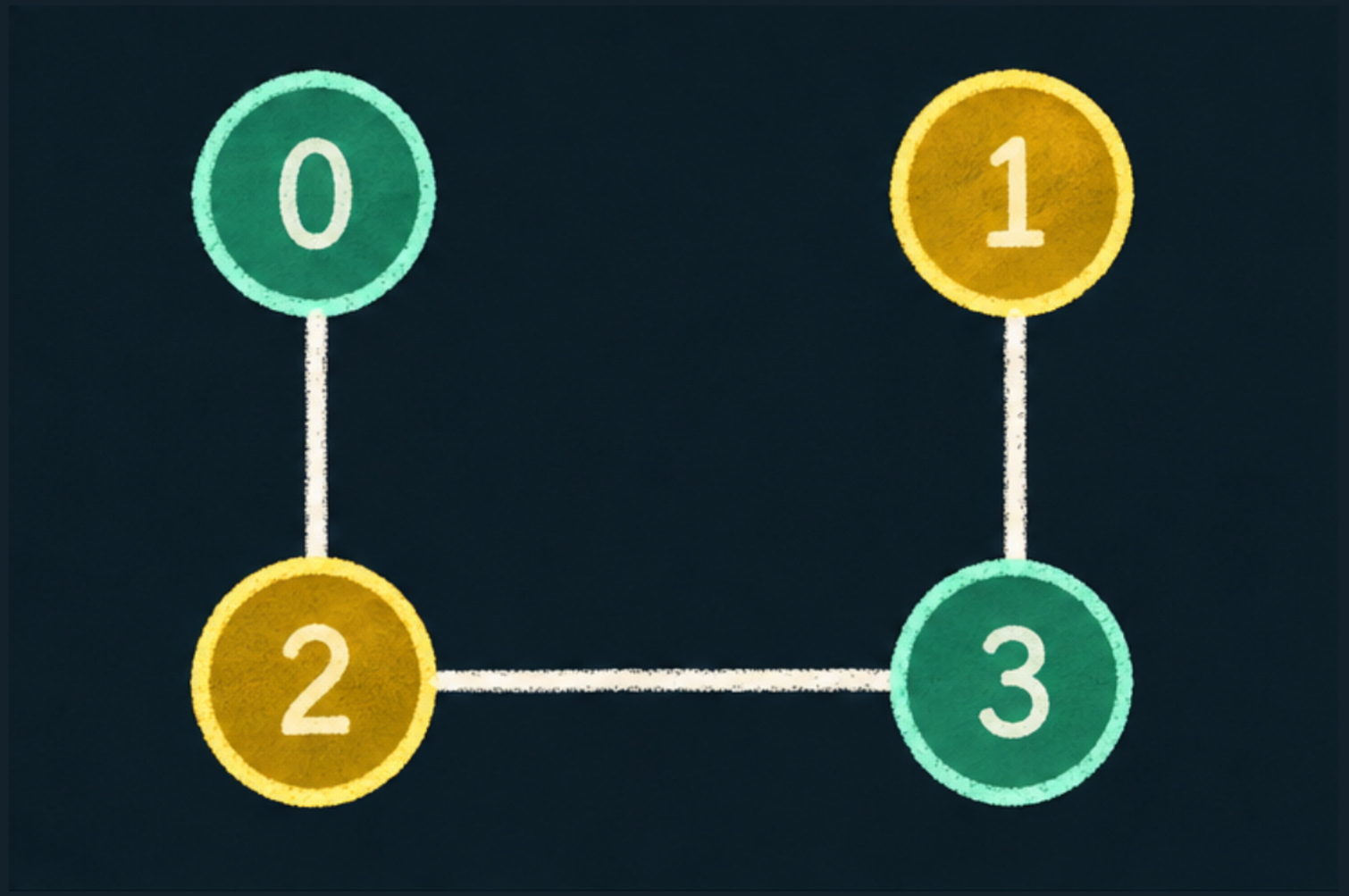
0번은 그대로 두고 1번만 바꿉니다.

2번은 0번과 다르게 노랑을 씁니다.

이제 3번의 두 이웃이 모두 노랑입니다.

3번에 초록을 칠할 수 있습니다.

모든 점을 칠한 방법 하나를 찾았습니다



색은 [초록, 노랑, 노랑, 초록]입니다.

세 선의 양 끝 색이 모두 다릅니다.

네 점을 하나도 빠짐없이 칠했습니다.

가능하므로 답은 1입니다.

코드에서는 색을 숫자로 적습니다

color=[0,0,0,0]으로 시작합니다. 0은 아직 칠하지 않았다는 표시입니다.

초록을 1, 노랑을 2로 정하면 완성 그림은 color=[1,2,2,1]입니다.

paint(v)는 v번 점부터 칠하는 함수입니다. 처음에는 paint(0)입니다.

다시 고를 때는 방금 적은 color[v]를 0으로 지웁니다.

현재 색을 적고, 다음 점을 칠해 봅니다

```
color[v] = c
if paint(v + 1) == 1:
    return 1
color[v] = 0
```

이번 색 c가 허용되는지 먼저 검사합니다.

다음 점부터 성공하면 현재 함수도 1을 반환합니다.

다음 점들이 실패했을 때만 이번 색을 0으로 지웁니다.

모든 후보 색이 실패하면 현재 함수는 0을 반환해 앞 점으로 돌아갑니다.

7-3 Python (앞부분)

```python
def can_color(graph, k):
    n = len(graph)
    color = [0]*n
    def paint(v):
        if v == n:
            return 1
        for c in range(1, k+1):
            allowed = True
            for u in range(v):
                if graph[v][u] == 1 and color[u] == c:
                    allowed = False
                    break
```

이어서 다음 화면의 코드를 붙여 같은 함수를 완성합니다.

7-3 Python (뒷부분)

```
13              if allowed:
14                  color[v] = c
15                  if paint(v+1) == 1:
16                      return 1
17                  color[v] = 0
18          return 0
19      return paint(0)
```

앞 화면에서 검사한 뒤, 색을 적고 다음 점으로 갑니다.

C 함수에서 n은 실제 점 수입니다

graph는 연결 표, n은 점 수, k는 쓸 수 있는 색 수입니다.

graph[][8]은 한 행의 저장 공간이 8칸이라는 C의 표시입니다. 앞의 n행·n열만 읽습니다.

color[8]={0}은 색을 저장할 공간 8칸을 모두 0으로 준비합니다.

입력 준비와 main 함수는 사이트가 맡습니다. 정해진 함수와 보조 함수를 작성합니다.

7-3 C (앞부분)

```
1   static int paint(const int g[][8], int n, int k,
2                    int v, int color[]) {
3       if (v==n) return 1;
4       for (int c=1; c<=k; ++c) {
5           int ok=1;
6           for (int u=0; u<v; ++u)
7               if (g[v][u] && color[u]==c) ok=0;
```

이어서 다음 화면의 코드를 붙여 같은 함수를 완성합니다.

7-3 C (뒷부분)

```
8             if (ok) { color[v]=c;
9                 if (paint(g,n,k,v+1,color)) return 1;
10                color[v]=0; }
11        } return 0;
12    }
13    int can_color(const int graph[][8], int n, int k) {
14        int color[8]={0}; return paint(graph,n,k,0,color);
15    }
```

앞 화면에서 검사한 뒤, 색을 적고 다음 점으로 갑니다.

두 문제는 같은 순서로 찾고, 답을 다르게 모읍니다

7-2는 가능한 선택의 수를 묻습니다. 각 갈래에서 찾은 성공 수를 모두 더합니다.

7-3은 가능한지 묻습니다. 모든 점을 칠한 방법 하나를 찾으면 1을 반환합니다.

두 문제 모두 하나를 시험하고, 필요하면 앞의 선택으로 돌아와 다른 것을 봅니다.

문제에서 구하라는 답을 먼저 읽고, 마지막에 무엇을 반환할지 정합니다.